solution to this problem was developed for the Waterken server that used capability URIs extensively, under the name *web-keys* (http://waterken.sourceforge.net/web-key/). In a web-key the unguessable token is stored in the fragment component of the URI; that is, the bit after a # character at the end of the URI. The fragment is normally used to jump to a particular location within a larger document, and has the advantage that it is never sent to the server by clients and never included in a `Referer` header or `window.referrer` field in JavaScript, and so are less susceptible to leaking. The downside is that because the server doesn't see the token, the client must extract it from the URI and send it to the server by other means.

In Waterken, which was designed for web applications, when a user clicked a web-key link in the browser, it loaded a simple template JavaScript page. The JavaScript then extracted the token from the query fragment (using the `window.location.hash` variable) and made a second call to the web server, passing the token in a query parameter or header (such as the Authorization: Bearer header you've used since chapter 5). The flow is shown in figure 9.3.

Because the JavaScript template itself contains no sensitive data and is the same for all URIs, it can be served with long-lived cache-control headers and so after the browser has loaded it once, it can be reused for all subsequent capability URIs without an extra call to the server, as shown in the lower half of figure 9.3. This approach works well with single-page apps (SPAs) as they often already use the fragment in this way to permit navigation in the app without causing the page to reload while still populating the browser